

The Campus Library Database Design Challenge

Week 9 Advanced Activity · a higher-order thinking task

Analyze

Create

Evaluate

THE SITUATION

You have inherited a broken database

The campus library runs on a single spreadsheet-turned-table that one well-meaning staff member built years ago. It works, barely. Records are duplicated, updates go wrong, and nobody can answer simple questions without scrolling. You have been hired to **diagnose it, redesign it, query it, and defend your decisions**.

This is not a fill-in-the-blanks exercise. Most questions have **more than one defensible answer** — you are graded on your reasoning as much as your result. Every part is tagged with the kind of thinking it demands.

The one table you inherited — `circulation`

Every row is one loan. Everything the library knows is crammed into this one table. Here is a sample.

member_id	member_name	member_email	isbn	title	genre	authors	author_emails	borrowed_on	due_on
1	Ana Cruz	ana@u.edu	978-1	Web Systems	Tech	Cruz; Santos	cruz@p.edu; santos@p.edu	2026-06-01	2026-06-15
1	Ana Cruz	ana@u.edu	978-2	Databases	Tech	Reyes	reyes@p.edu	2026-06-03	2026-06-17
2	Ben Lim	ben@u.edu	978-1	Web Systems	Tech	Cruz; Santos	cruz@p.edu; santos@p.edu	2026-06-05	2026-06-19
2	Ben Lim	ben@u.edu	978-3	Networks	Tech	Reyes	reyes@p.edu	2026-06-08	2026-06-22

Assume, for now, that a member borrows a given title at most once, so the key of this table is `(member_id, isbn)`.

How you will be judged

- **Correctness** — the SQL runs, the schema holds, the keys are right.
- **Reasoning** — you can explain *why*, name the principle, and show your steps.
- **Judgement** — on open questions you weigh trade-offs and commit to a defended choice.

Time

90 minutes

Work

Pairs; both must be able to defend every answer

Allowed

Your notes, the SQL and normalization references

Diagnose the flawed table

Before you fix anything, prove you understand what is wrong. Work from the `circulation` sample on the previous page.

1 Map the functional dependencies

Using formal notation ($X \rightarrow Y$), list every functional dependency you can find in `circulation`. Include the one that holds on the whole key and the ones that hold on only part of it.

2 Classify each dependency

Label each FD from Q1 as a **partial** dependency, a **transitive** dependency, or a **full** dependency on the key. For the transitive one, write the chain (e.g. `key  $\rightarrow$  A  $\rightarrow$  B`).

3 Expose the anomalies with evidence

Give a **concrete example from the sample data** of each of the three modification anomalies. Do not just define them — point to the actual rows and say what breaks.

Anomaly	Your concrete example from the data
Update	
Insertion	
Deletion	

4

Judge the current normal form

What is the **highest** normal form `circulation` currently satisfies? Justify your answer by naming the exact rule it *fails* for the next form up, and the specific column that violates it.

Redesign it properly

5 Normalize, showing every step

Decompose `circulation` all the way to **3NF**. Do not jump to the answer — show the table set after **1NF**, then **2NF**, then **3NF**, and next to each step name the dependency you removed to get there.

6 Model a many-to-many relationship

You assumed one author per book, but the data shows `Cruz; Santos` — a book can have several authors, and an author writes several books. A foreign key cannot express this. Design the tables that can, including the **junction table**. Write them as `CREATE TABLE` statements with keys.

Higher-order prompt

What is the primary key of your junction table, and why can it not be a single new `SERIAL` id if you want to stop the same author being linked to the same book twice?

Design from an ambiguous requirement

The head librarian says, in passing: *"We need to handle the fact that we own several physical copies of popular titles, and members should be able to reserve a title that is all checked out, in the order they asked."* That one sentence hides at least two design decisions. State the assumptions you are making, then design the schema change(s) to support it.

Query the redesigned database

Use your normalized tables from Part B (`members` , `books` , `authors` , `book_authors` , `loans`). Assume `loans` also has a `returned_on` column that is `NULL` until the book comes back.

8 Three-table join

List every loan showing the **member name**, the **book title**, and the **borrowed date** — joining across all three tables.

9 Aggregate with a condition

Find every member who currently has **more than 3 books on loan** (not yet returned), showing the member id and their count. You will need `GROUP BY` and `HAVING` — and explain in one line why `WHERE` alone cannot do this.

10 Subquery

List the titles of all books that have **never been borrowed**. Do it with a subquery (`NOT IN` or `NOT EXISTS`), and say which of those two you chose and why.

11 Compute overdue books

List every book that is **overdue right now** — past its due date and not yet returned — with the member who has it. Use `CURRENT_DATE` .

Trace the output

Without running it, state exactly what this query returns for the sample data, and what it is really asking. Then say what a single missing `WHERE` word would change.

what does this return?

```
SELECT m.member_name, COUNT(l.book_id) AS total
FROM members m
LEFT JOIN loans l ON l.member_id = m.member_id
GROUP BY m.member_name
ORDER BY total DESC;
```

Defend your engineering decisions

There is no single correct answer here. You are graded on the quality of your argument — name the principle, weigh both sides, and commit.

13

Normalized vs denormalized

A colleague proposes an extra `loan_report` table that repeats the member name, book title, and author names on every loan row — “so the dashboard needs no joins.” Give **one real advantage** and **one real cost** of this. Then recommend for or against it, and state the *usage pattern* that would change your recommendation.

14

SQL or NoSQL — and why

The library wants to log **every page-view and search click** on its catalog site — millions of events a day, where being off by a second or losing one event does not matter. Would you keep this in your PostgreSQL tables or reach for a NoSQL store? Justify your choice using **ACID vs BASE**. Then name one kind of library data where the opposite choice is clearly correct.

15

Set the condition, add the safeguard

Denormalizing for speed is sometimes right. State the **specific condition** under which you would deliberately break 3NF in this system, and the **one safeguard** you would put in place so the duplicated data cannot silently go wrong.

How this is graded

Weighted toward the higher-order tiers. Reasoning and judgement carry more than a correct-but-unexplained answer.

Part	What is assessed	Level	Pts
A	FDs mapped and classified correctly (Q1–Q2)	Analyze	12
A	Anomalies shown with real evidence; correct current NF justified (Q3–Q4)	Analyze	13
B	Stepwise 1NF→2NF→3NF with the removed dependency named at each step (Q5)	Create	15
B	Correct many-to-many design with a valid junction key (Q6)	Create	10
B	Ambiguous requirement: assumptions stated, workable schema (Q7)	Create	10
C	Join, aggregation+HAVING, subquery, overdue, trace (Q8–Q12)	Apply / Analyze	25
D	Trade-off, ACID/BASE, and safeguard arguments — principle named, both sides weighed, choice defended (Q13–Q15)	Evaluate	15
Total			100

What a top answer looks like

- **Analyze:** points at specific rows as evidence, not textbook definitions.
- **Create:** shows intermediate tables and can defend each key choice; realises the junction key is the composite `(book_id, author_id)`.
- **Evaluate:** commits to a decision and names the exact condition that would reverse it — a fence-sitting answer scores low even when both sides are listed.

Instructor answer key

A full worked solution is available separately. Quick keys: Q4 — the table is in **1NF at best** once `authors` / `author_emails` are made atomic, and fails 2NF because `member_name` and `title` depend on only part of the key `(member_id, isbn)`. Q6 — junction table `book_authors(book_id, author_id)` with a **composite primary key** of both FKs, which is what prevents duplicate links. Q12 — returns every member with their total loan count, highest first, and includes members with zero loans because of the `LEFT JOIN`; changing it to an inner `JOIN` would drop the zero-loan members.